```
    {
        public bool Match(HttpContext httpContext, IRouter route,          ❷
        ➥ string routeKey, RouteValueDictionary values, RouteDirection    ❷
        ➥ routeDirection)                                                 ❷
        {
            var cities = new[] { "amsterdam", "barcelona", "berlin",       ❸
                                 ➥ ”copenhagen”, “dubrovnik”, “edinburgh”,❸
                                 ➥ "london", "madrid", "paris", "rome",   ❸
                                 ➥ "venice" };                            ❸
            return cities.Contains(values[routeKey]?.ToString()
➥ .ToLowerInvariant());                                                   ❹
        }
    }
}
```

❶ The constraint must implement IRouteConstraint.

❷ The IRouteConstraint specifies a Match method that returns a Boolean.

❸ The array of valid values

❹ The code that determines if the match is valid.

The `IRouteConstraint` interface has one member—a method named `Match` that returns a Boolean. It takes a number of items as parameters, not all of which will be needed in order to perform the match in every case. In this example, only the `RouteValuesDictionary` and the `routeKey` are needed. The `routeKey` value is the name of the parameter to be checked. If there is a match between the incoming parameter value and an item in the cities array, the `Match` method returns `true`.

NOTE The `Match` method in `IRouteConstraint` is a synchronous method, which makes `IRouteConstraint` unsuitable for any requirement that should involve asynchronous processing. If you want to constrain incoming routes in a real-world application asynchronously, alternative mechanisms that support this include middleware (see chapter 2) and `IEndPointSelectorPolicy` (https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.routing.matching.iendpointselectorpolicy?view=aspnetcore-7.0), which is not covered in this book.

Custom route constraints must be registered. This is done in the `Program` class. The constraint is added to the `ConstraintMap` collection, as